

```
using System;
```

```
namespace ConsoleApp4
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        enum DayOfWeek
```

```
        {
```

```
            Monday = 1,
```

```
            Tuesday = 2,
```

```
            Wednesday = 3,
```

```
            Thursday = 4,
```

```
            Friday = 5,
```

```
            Saturday = 6,
```

```
            Sunday = 7
```

```
        }
```

```
        static void Main(string[] args)
```

```
        {
```

```
            #region p1
```

```
            //int[] arr1 = new int[3];
```

```
            //arr1[0] = 1;
```

```
            //arr1[1] = 2;
```

```
            //arr1[2] = 3;
```

```
            //int[] arr2 = new int[] { 1, 2, 3 };
```

```
            //int[] arr3 = { 1, 2, 3 };
```

```
            //Console.Write("arr1 ");
```

```
            //for (int i = 0; i < arr1.Length; i++)
```

```
            //{
```

```
            //    Console.Write(arr1[i] + " ");
```

```
            //}
```

```
            //Console.Write("arr2 ");
```

```
            //for (int i = 0; i < arr2.Length; i++)
```

```

//{

// Console.Write(arr2[i] + " ");

//}


//Console.Write("arr3 ");

//for (int i = 0; i < arr3.Length; i++)

//{

// Console.Write(arr3[i] + " ");

//}


//try

//{

// Console.WriteLine(arr1[5]);

//}

//catch (IndexOutOfRangeException ex)

//{

// Console.WriteLine( ex.Message);

//}

// //default values Numeric 0,Boolean false,Reference null,Character '\0'

#endregion


#region p2

//int[] arr1 = { 10, 20, 30 };

//int[] arr2 = arr1;


//Console.WriteLine(arr1[0]);

//arr2[0] = 9;

//Console.WriteLine(arr1[0]);


//int[] arr3 = { 1, 2, 3 };

//int[] arr4 = (int[])arr3.Clone();

//Console.WriteLine(arr3[0]);

//arr4[0] = 5;

//Console.WriteLine(arr3[0]);

//Console.WriteLine(arr4[0]);

////Array.Clone Creates a new array object with the exact same size and copies all elements into it.

////Array.Copy() Copies a range of elements from a source array to an already existing destination array.

```

```
#endregion
```

```
#region p3
```

```
//int[,] grades = new int[3, 3];
```

```
//for (int i = 0; i < 3; i++)
```

```
//{
```

```
//  Console.WriteLine($"Enter grades for Student {i + 1} ....");
```

```
//  for (int j = 0; j < 3; j++)
```

```
//  {
```

```
//      Console.Write($"Subject {j + 1}");
```

```
//      grades[i, j] = int.Parse(Console.ReadLine());
```

```
//  }
```

```
//}
```

```
//for (int i = 0; i < grades.GetLength(0); i++)
```

```
//{
```

```
//  Console.Write($"Student {i + 1}");
```

```
//  for (int j = 0; j < grades.GetLength(1); j++)
```

```
//  {
```

```
//      Console.Write(grades[i, j] + "\t");
```

```
//  }
```

```
//}
```

```
////Length Returns the total number of elements
```

```
////GetLength(int dimension Returns the number of elements in a specific dimension row or coulumn
```

```
#endregion
```

```
#region p4
```

```
//int[] numbers = { 40, 10, 50, 20, 30 };
```

```
//Array.Sort(numbers);
```

```
//// 10 20 30 40 50
```

```
//Array.Reverse(numbers);
```

```
////50, 40, 30, 20, 10
```

```

//int index = Array.IndexOf(numbers, 20);

///< 3

//int[] copyArray = new int[5];
//Array.Copy(numbers, copyArray, numbers.Length);
///< نسخ كل عناصر المصفوفة

//Array.Clear(numbers, 0, numbers.Length);
///< تصفير كل عناصر المصفوفة 0

///

```

```

#region p6

//int inputNumber;

//bool isValidInput;


//do

//{

//  Console.WriteLine("Enter an odd number: ");

//  isValidInput = int.TryParse(Console.ReadLine(), out inputNumber);


//  if (!isValidInput)

//  {

//    Console.WriteLine("Invalid input! Please enter a valid integer.");

//  }


//} while (!isValidInput || inputNumber % 2 != 0);

////Prevents Program Crashes,Enhances Application Security

#endregion

```

```

#region p7

//int[,] matrix = {

//  { 10, 20, 30 },

//  { 40, 50, 60 },

//  { 70, 80, 90 }

//};


//for (int i = 0; i < matrix.GetLength(0); i++)

//{

//  for (int j = 0; j < matrix.GetLength(1); j++)

//  {

//    Console.Write($"{matrix[i, j]}\t");

//  }

//  Console.WriteLine();

//  //Inserts fixed tab spaces between column elements

//  //Console.WriteLine() after the inner loop add new line

#endregion

```

```

#region p8

```

```

//Console.WriteLine("Enter a month number: ");

//if (int.TryParse(Console.ReadLine(), out int monthNumber))

//{

//  if (monthNumber == 1) Console.WriteLine("January");

//  else if (monthNumber == 2) Console.WriteLine(" February");

//  else if (monthNumber == 3) Console.WriteLine("March");

//  else if (monthNumber == 4) Console.WriteLine(" April");

//  else if (monthNumber == 5) Console.WriteLine(" May");

//  else if (monthNumber == 6) Console.WriteLine("June");

//  else if (monthNumber == 7) Console.WriteLine("July");

//  else if (monthNumber == 8) Console.WriteLine("August");

//  else if (monthNumber == 9) Console.WriteLine("September");

//  else if (monthNumber == 10) Console.WriteLine(" October");

//  else if (monthNumber == 11) Console.WriteLine(" November");

//  else if (monthNumber == 12) Console.WriteLine("December");

//  else Console.WriteLine("Invalid month number Please enter a number between 1 and 12");

//}

//else

//{

//  Console.WriteLine("Invalid input! Please enter a valid integer.");

//}

```

```

//Console.WriteLine("Enter a month number ");

//if (int.TryParse(Console.ReadLine(), out int monthNumber2))

//{

//  switch (monthNumber2)

//  {

//    case 1: Console.WriteLine("January"); break;

//    case 2: Console.WriteLine("February"); break;

//    case 3: Console.WriteLine("March"); break;

//    case 4: Console.WriteLine("April"); break;

//    case 5: Console.WriteLine(" May"); break;

//    case 6: Console.WriteLine("June"); break;

//    case 7: Console.WriteLine("July"); break;

//    case 8: Console.WriteLine(" August"); break;

```

```
// case 9: Console.WriteLine(" September"); break;

// case 10: Console.WriteLine("October"); break;

// case 11: Console.WriteLine("November"); break;

// case 12: Console.WriteLine(" December"); break;

// default:

// Console.WriteLine("Invalid month number!");

// break;

// }

//}
```

لما الشروط تزيد عن 5 حالات

```
#endregion
```

```
#region p9
```

```
//int[] numbers = { 42, 15, 8, 23, 15, 8, 90, 15, 4 };
```

```
//Array.Sort(numbers);
```

```
//int target = 15;
```

```
//int firstIndex = Array.IndexOf(numbers, target);
```

```
//int lastIndex = Array.LastIndexOf(numbers, target);
```

```
//// n log n
```

```
#endregion
```

```
#region p10
```

```
//int[] numbers = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100 };
```

```
//int sumFor = 0;
```

```
//for (int i = 0; i < numbers.Length; i++)
```

```
//{
```

```
// sumFor += numbers[i];
```

```
//}
```

```
//Console.WriteLine($"{sumFor}");
```

```
//int sumForeach = 0;
```

```
//foreach (int num in numbers)
```

```
//{
```

```
// sumForeach += num;
```

```
//}
```

```

//Console.WriteLine($"{sumForeach}");

////for ل تنفيذ
//for ل تنفيذ
//foreach كقرايه وحمايه من الاخطاء

#endregion

#region enum

//Console.Write("Enter a number (1-7):");

//int input = int.Parse(Console.ReadLine());

//DayOfWeek day = (DayOfWeek)Enum.Parse(typeof(DayOfWeek), input.ToString());

//Console.WriteLine($"Day:{day}");

//// التحويل هينجح بدون أخطاء، والمتغير هيخزن الرقم عادي ولما اجي اطبعه مش هيكون متخزنله قيمه اسم
#endregion

}

}

}

```

وأنا بتعلم #C اتعرفت على أنواع مختلفة من الـ Loops، وكل نوع بيكون مناسب لموقف معين.

الفرق الأساسي بينهم هو إمتى نستخدم كل واحد.

for Loop

بنستخدمها لما نكون عارفين عدد مرات التكرار.

مثال:

```
for (int i = 0; i < 5; i++)
```

```
}
```

```
;Console.WriteLine(i)
```

```
{
```

مناسبة لما أقول مثلاً "كرر العملية 5 مرات".

foreach Loop

بنستخدمها لما نكون عايزين نمر على كل عنصر في Collection أو Array.

مثال:

```
foreach (string name in names)
```

```
}
```

```
;Console.WriteLine(name)
```

```
{
```

مناسبة لما أتعامل مع Array أو List ومش محتاج أتحكم في الـ Index.

while Loop

بنستخدمها لما عدد مرات التكرار مش معروف، والتكرار يعتمد على Condition.

```
while (number < 10)
```

```
}
```

```
;++number
```

```
{
```

الـ Condition بيتفحص الأول، ولو كان False من البداية، الـ Loop مش هتنتفذ.

do-while Loop

شبه الـ while، لكن الفرق إن الكود بيتنفذ مرة واحدة على الأقل قبل فحص الـ Condition.

```
do
```

```
}
```

```
;++number
```

```
{
```

```
;while (number < 10)
```

مناسبة لما أحتاج العملية تحصل مرة واحدة على الأقل، وبعدها أقرر أكمل ولا لأ.

باختصار:

for → لما أعرف عدد التكرارات.

foreach → لما ألف على عناصر Collection.

while → لما التكرار يعتمد على Condition ويمكن مايتنفذ أصلاً.

do-while → لما لازم يتنفذ مرة واحدة على الأقل.

فهم الفرق بينهم بيساعدني أختار الـ Loop المناسبة بدل ما أستخدم نفس النوع في كل موقف.

CSharp #DotNet #Programming #Loops #SoftwareDevelopment #DEPI #LearningJourney#

S1

When creating an Array of Reference Types (Objects), memory allocation happens in two steps:

Array Allocation: Constructing the array creates a contiguous block in the Heap that holds references (pointers) initialized to null, rather than the actual object data.

Object Instantiation: Each element in the array must be explicitly instantiated using the new keyword. This allocates memory for the actual object in the Heap and assigns its address to the corresponding array slot.

Unlike Value Type arrays where values are stored directly inside the array, an Array of Objects is an array of memory addresses pointing to objects elsewhere in the Heap.

S2

A Jagged Array is a multi-dimensional structure implemented as an array whose elements are themselves arrays.

Key Characteristics:

Non-Uniform Dimensions: Unlike rectangular arrays, each row in a jagged array can have a different length or number of columns (No Relational structure).

Memory Allocation: The main array stores references to sub-arrays in the Heap. Each child array is allocated independently in memory.

Declaration Syntax: It is declared using multiple sets of square brackets, such as int[][].

Flexibility: It is ideal for storing grouped data where rows do not require equal lengths, optimizing memory usage by avoiding empty, unused elements.

الـ Stack والـ Heap هما المكانين اللذين يخزن فيهما البيانات في الذاكرة:

الـ Stack:

هذا المكان السريع المباشر في الذاكرة. مميزاتة إنه سريع جداً في الحجز والمسح بمجرد ما الـ Method تخلص، ويخزن فيه الـ Local Variables والـ Value Types. بس عيبه إن حجمه محدود أوي، فلو عملنا Loop مقفولة أو Recursion ملوش نهاية بيضرب StackOverflowException على طول.

الـ Heap:

هذا المكان المفتوح والكبير في الذاكرة ومش واخد حجم ثابت من الأول، بيوسع ويتشد على حسب محتاجين إيه ومساحة الـ RAM المتاحة في الجهاز كام. يخزن فيه الـ Reference Types والأوبجكتس الكبيرة. الميزة هنا إن الـ Garbage Collector هو اللذي بيتولى تنظيفه من الأوبجكتس اللذي ملهاش لازمة تلقائياً. وعيبه إن التعامل معاه أبطأ شوية من الـ Stack ولو البرنامج زاد عن حده أوي ممكن يضرب OutOfMemoryException.

Time complexity measures how the execution time of an algorithm increases as the size of the input data grows. Instead of measuring time in actual seconds (which varies depending on your computer's processor or RAM), time complexity counts the number of basic operations an algorithm performs.